

3. Network Security

Sniffing 스니핑

- 중간에 훑쳐보기
- 네트워크를 통해 전송되는 모든 data packet을 지속적으로 모니터링하거나 기록하는 것
- sniffer를 활용하여 데이터 패킷을 캡처함. such as 민감 정보, 비밀번호, 계정정보

Spoofing 스푸핑

- 속이다. 사기치다
- 대상의 개인정보에 액세스하거나, 감염된 링크, 첨부파일을 통해
>> 멀웨어 전파, 네트워크 제어 우회, DOS 공격을 위한 redistribute traffic
- rerouting으로 이어지면(다른 사이트로 연결)되면 네트워크 과부하, 정보 탈취, 멀웨어 배포사이트로 연결됨

DOS(Denial of Service) attack

- 네트워크 또는 서버에 요청, 데이터를 폭주시켜 컴퓨터 리소스를 사용할 수 없게 하는 공격(Availability 저해)
 - * 디스크, 데이터, 시스템
 - * 사용가능한 컴퓨팅 자원(하드, 메모리, CPU 시간) 전부 소비
 - * 엄청난 양의 패킷으로 인한 네트워크 대역폭 고갈

Vulnerability attack(Boink Bonk Teardrop)과 Resource exhaustion attack(LANd, Flooding...)으로 나뉨.

Sniffer가 허브와 스위치에서 작동하는 방법

Hub(공유 이더넷 네트워크)

- * C1 -> C2로 가지만 C3, C4 도 받음
- * 보통은 C3, C4는 패킷을 reject함. MAC 주소가 안맞기 때문
- * Promiscuous mode(무차별 모드), C3, C4의 전체 통신 캡처

Switch(전환된 네트워크)

- * C1 -> C2로 이동하는 패킷은 스위치에 연결된 다른 컴퓨터에서 수신되지 않음.

Sniffing 공격의 종류

Port mirroring(SPAN : Switch Port analyzer - CISCO)

네트워크 스위치에서 한 포트에서 발생하는 네트워크 패킷을 복사하여 다른 포트에 전송하는 기능
IDS와 같이 네트워크 트래픽을 모니터링해야할 때 사용됨.

>> 해커가 특정 포트에 대한 사용자 권한을 얻어 공격할 수 있음.

Switch jamming(스위칭 방해)

1. 대량의 가짜 매핑 정보(MAC포트)를 스위치에 지속적으로 전송
2. MAC 주소 테이블이 overflow됨
3. 스위치 기능이 마비되고, dummy hub처럼 broadcast를 한다.
4. 공격자는 스위치에서 방출되는 정보를 sniffing 할 수 있음.

Sniffing 감지와 방어

감지 방법

- Ping method : ICMP(Internet Control Message Protocol)를 보내 네트워크 트래픽을 몰래 캡처하고 분석.
의심되는 주소의 IP 주소로 전송. but NOT 그 MAC주소
- ARP method : ARP watch라는 유틸리티가 기계 복제가 있는지 ARP 캐시들을 모니터.
이더넷 주소가 바뀌거나 IP와 MAC이 중복되면 문제가 있다!
- Local host method(ipconfig) : 로컬 시스템(UNIX, LINUX)에서 promiscuous(무차별) 모드인지 체크

방지 방법

- * 모든 들어오고 나가는 데이터를 **암호화**하는 것이 효과적인 방법
- * 안전하지 않은 네트워크 사용 피하기

Spoofing의 여러 방법

- 이메일 Spoofing : 살짝 바꾼 이메일 주소로 메일을 보내 정보를 훔치고 멀웨어 감염시킴
- Text message(SMS) Spoofing : 다른 사람의 전화번호로 속이는 문자를 보내는 것(Smishing)
- URL Spoofing : 악성 웹사이트를 자주 방문하는 사이트인 것처럼 꾸며 로그인 페이지처럼 만드는 것
- Man-in-the-middle attack
intercepting : 공격자가 무료 Wifi 핫스팟을 사용할 수 있게 함
이 공격은 IP, ARP, DNS Spoofing보다도 더 많은 일들을 할 수 있음.

ARP(Address Resolution Protocol) Spoofing

LAYER 2

: LAN을 통해서 위조된 메시지를 보내는 Man-in-the-middle 공격임.

Aka ARP Poisoning

작동방식

1. 공격자가 네트워크를 스캔하여 최소 2개 장치(PC, 라우터)의 IP주소를 결정한다.
2. 공격자가 스푸핑 도구를 사용해 위조된 ARP 응답을 보냄
3. 위조된 응답은 라우터와 PC의 IP주소에게 대한 올바른 MAC 주소가 공격자의 MAC 주소라고 알린다.
4. 두 디바이스가 ARP 테이블을 업데이트하고, PC와 라우터가 서로 통신하는 대신 공격자와 통신하게 된다.
5. 공격자는 Man-in-the-middle을 통해 라우터와 PC간 통신을 가로채고 수정하거나 기록할 수 있음

감지방식

'arp'라는 명령어로 ARP 테이블을 보기 - ARP watch 사용
다른 2개의 IP주소가 하나의 MAC 주소를 가리키고 있으면 ARP 공격이 발생하고 있음을 나타냄

IP Spoofing

: 다른 컴퓨팅 시스템으로 가장하기 위해 잘못된 IP 주소의 IP 패킷을 만드는 것.
>>IP 주소를 다른 것으로 속여 Respond를 다른 주소로 하게 만드는 것

IP address trusted authentication : unauthorized한 호출을 차단하고 Cisco Unified CME 시스템을 잠재적인 통화 사기와 같은 부정사용으로부터 보호합니다. >> 문제점 **IP Spoofing**에서는 더 큰 문제점이 될 수 있음.

방지 방법

Packet Filtering : 외부에서 들어온 request가 외부로 다시 나가는 것이 아니라 내부에서 처리가 되네?
>>Ingress filtering(입구 필터링)

Trusted Authentication 같은 방식 **사용하지 말기**
지속적인 관리와 감시

ICMP(Internet Control Message Protocol) Spoofing

ICMP의 네트워크 제어 기능을 악용하는 공격

시스템의 취약점을 이용해 원격 공격자가 시스템 메모리 누수를 일으키고 DoS를 초래할 수 있음

ICMP Redirect message : 호스트에게 최적의 경로를 알려주기 위해 설계된 메시지

>> 공격자가 악용하여 트래픽을 특정 시스템으로 우회시킬 수 있음.

방지 방법

ICMP Redirect Acceptance를 비활성함 (in Linux),
'/etc/sysctl.conf' file and add the following line:

net.ipv4.conf.all.accept_redirects = 0

<< 1에서 0으로 바꾸면 원천적으로 방지 가능

DNS(Domain Name Service) Spoofing

:변경된 DNS 레코드를 사용하여 원래 사이트와 유사한 사기 사이트로 리디렉션 하는 공격
aka DNS cache poisoning

사용자의 계정정보나 기타 민감한 정보가 유출될 수 있음.

방지 방법

DNSSEC(DNS security) 사용

DNS 소유자가 DNS 항목을 설명하면 DNSSEC이 암호 서명(cryptographic signature)을 추가한다.

hosts 파일에 ip 주소를 등록하면 방지 가능

DOS attacks

DOS(Denial of Service)

1개의 컴퓨터와 1개의 인터넷 연결 1대를 사용하여 대상 시스템이나 리소스를 flood함
시스템이 쉽게 중지되거나/protected 될 수 있음
malware 없음

DDOS(Distributed Denial of Service)

여러 컴퓨터와 인터넷을 이용하여 대상 시스템이나 리소스를 flood함
공격을 막기가 어려움
감염된 botnet(zombie PC)를 이용해 공격함.

Vulnerability attacks

Boink, Bonk, Teardrop

- * TCP 프로토콜의 약점을 이용한 공격
- * 대상이 반복적인 요청과 수정을 하도록 해 강제적으로 시스템 리소스를 고갈시킴

Bonk : 시퀀스 번호를 조작하여 계속 첫 번째 것을 보내는 것처럼 함. 계속 connection 유지
첫 번째 패킷을 보낼때도 1, 두 번째, 세 번째 패킷을 보낼 때도 1을 보냄. 계속 1번만 오는 줄 앎.
Boink : 일정한 시퀀스 번호를 가진 패킷을 **중간에 삽입**. 시스템 혼란, 과부하
첫 번째는 정상적으로 전송하고 이후 패킷은 일정한 시퀀스 번호를 가지며, 중간에 삽입.
Teardrop attack : 조각화된 IP 패킷을 중첩되게 전송하여 다시 조립이 불가능하게 만들.

많은 양의 데이터가 인터넷을 통해 전송되기 전에 조각화됨
각 fragment에 특정 번호가 할당되고 MTU : 1500바이트), 수신이 다 되면 fragment를 재배열하여 원래 메시지를 재구성함.
여기서 Teardrop 공격이 Fragment offset 값을 조작하여 제대로 reassemble하지 못하게 만들.
엄청난 수의 패킷이 누적되어 충돌을 일으킴.

방지 방법 : 방화벽을 사용해서 들어오는 패킷의 프레임 정렬과 잘못 포맷된 패킷들을 버린다.

LAND(LAN Denial) attack

호스트 또는 해당 IP stack을 손상시키거나 비활성화 시키는 것
주소/포트 페어를 자기 자신에게 연결
패킷 소스 주소를 spoof 할 수 있어야 함.
외부에서 오는 공격을 필터링을 통해서 막아야 함.

Ping of death

ping 명령어를 사용하여 형식이 잘못되었거나 크기가 큰(>65535) 패킷을 전송하여 대상 컴퓨터/서비스를 충돌, 불안정 또는 정지시키는 공격

방지 방법

방화벽을 통해선 ICMP Ping 메시지를 차단해버리기
단편화된 핑을 선택적으로 차단, 실제 핑이 방해받지 않고 통과할수 있게 하기

SYN Flooding(Half-open attack)

TCP 호스트에 대한 액세스를 거부하는 것. DOS의 일종

SYN > SYN/ACK > ACK > 종료

호스트가 의도적으로 **ACK 패킷을 보내지 않아** 연결이 완료되지 않은 상태로 있음. > 자원 소진

방지 방법

서버의 queue 사이즈를 늘린다.
서버의 최대 연결 latency를 줄인다
IPS 설치, 최신 네트워크 장비, 모니터링 기구 사용

ICMP Flooding

ICMP echo request(ping)을 victim에게 엄청나게 많이 보내서 장치를 마비시키는 공격
echo request를 broadcast로 많은 장치에게 보낸 후, victim이 response를 받도록 하는 방법
aka **Smurf Attack, Ping flood**

UDP flooding

UDP는 포트를 통해서 통신

대량의 UDP 패킷을 목표 시스템으로 보내는 것. UDP는 연결을 하지 않아 매우 빠르고 대량으로 전송 가능
서버가 UDP 패킷을 수신하면 OS가 해당 애플리케이션이 있는지 확인함.

앱이 없으면 발신자에게 에러 메시지를 보내야 함.

공격자가 발신자 IP 주소 스푸핑(속이기)해서 패킷을 보내서 오류 메시지를 무작위 컴퓨터로 전송됨

HTTP GET Flooding

HTTP 요청을 엄청나게 많이 보냄. DDOS 방식임

HTTP POST attack : HTTP Header Content-type/Content-Length을 조작. 이미지 유형, 비디오 유형으로 하고 Content-length를 1억바이트 같이 엄청 큰 숫자를 지정하면 리소스를 잡아먹게 함

Dynamic HTTP Request Flooding : HTTP 요청의 접미사를 바꿔서 계속 요청함.

HTTP Header DOS attack : HTTP 헤더에 CR(줄바꿈), LF(라인피드)... 문자들 없이 전송해버림

>>에러가 계속 생겨 계속 다시 요청함

HTTP CC(Cache-Control) attack

캐시 옵션을 no-store(저장안함), must-revalidate(반드시새로접속)으로 설정해 리소스 소모를 유도..

Billion laugh attack

aka XEE or XML Bomb

재귀적으로 entity를 확장할 때 과도한 메모리 소모 초래

중첩된 재귀 함수를 악용

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE foo [ <!ENTITY lol "lol">
<!ENTITY lol1 "&lol;&lol;&lol;&lol;">
<!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;"> ]>
<foo>&lol2;</foo>
```

Regular expression denial of service (ReDoS)

Regular expression이 처리가 오래 걸리게 하는 것 입력

regex = /A(B|C+)+D/

The expression would match inputs such as
ABBD, ABCCCD, ABCBCCD and ACCCCD

but it may take so much time to validate a string
like ACCCCCCCCCCCCCCCCCCCCCCCCCCCCCCX

A : String must start with 'A'
(B|C)+ : The string must then follow the letter
A with either the letter 'B' or some number of
occurrences of the letter 'C' (the + matches one
or more times)
D : This section of the string ends with a 'D'

>>시간을 오래 걸리게 함.

4. Web Security 1-1

tag : HTML 요소의 시작과 끝을 나타냄

can't be used overlapping each other

<H1> content </H1> (X)

<H1> content </H1> (O)

<head>

<script> 스크립트 </>

<style> 스타일정보 CSS </>

<meta> 메타데이터 기록

<title> 페이지 타이틀

<link> 링크 </>

</head>

<body bgcolor="#0000FF" text="FFFF00">

// "#000000"< 16진수 표현이다! 두자리씩 8비트 RGB 세묾음> 24비트. 투명도 알파채널 포함하면 32비트

<p align="left/right/center"> 문단 태그 </p>

➤ Bold Tag

➤ Italics Tag <I></I>

➤ Strike Tag <STRIKE></STRIKE> 가운데줄

➤ Small Tag <SMALL></SMALL> 작게

➤ Big Tag <BIG></BIG> 크게

➤ Subscript Tag 아래첨자

➤ Superscript Text Formatting 위첨자

➤ Teletype Text Tag <TT></TT> 타자기처럼

Font Tags • attribute: face (face = 'Arial') color (color=' #00FF00') size (size = '10')

➤ Heading Tags

• H1 (bigger) ~ H6 (smaller)

 Anchor tag 클릭하면 사이트로 이동

• name: 웹페이지 내의 특정 위치를 지정함.

page for this Hyperlink to reach when clicked

• target: defines where the URL defined with href attribute will open

_blank : 새로운 페이지/탭에서 열기

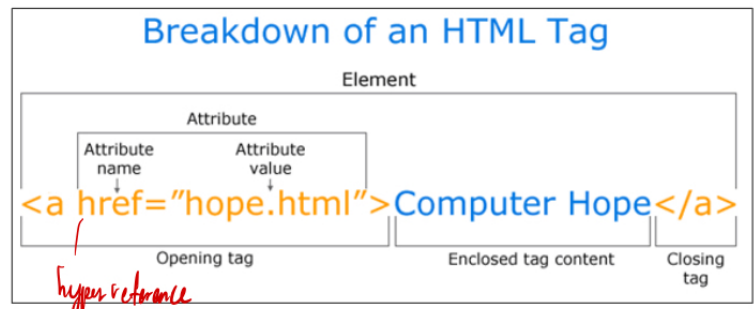
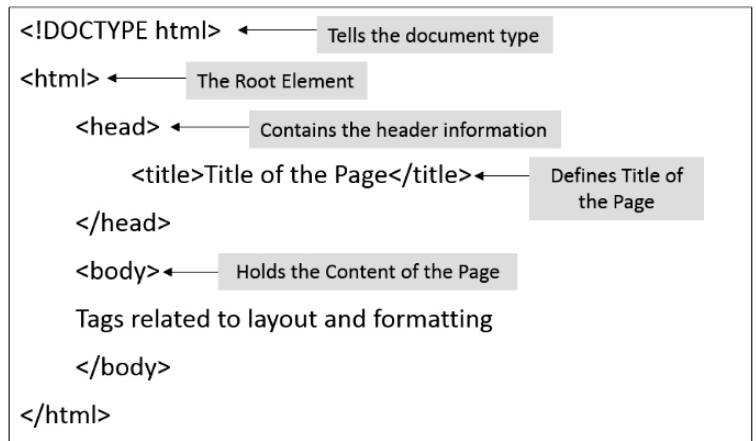
_self : (default) 같은 공간에서 페이지 열기

_parent : 부모 프레임에서 열기

_top : 가장 위에서 열기

filename : 해당 이름의 frame에서 열기

• type: 링크의 MIME(데이터 전송) 타입을 지정합니다. 보통 파일 다운로드 링크에서 사용됨



▪ HTML4.01 - Image tag

➤ is used to insert images in a webpage

➤ attribute

- src: 경로
- alt: 대체텍스트
- align: 정렬 LEFT (default), RIGHT, TOP, BOTTOM or MIDDLE
- border: 테두리
- width: 픽셀이나 %로 크기지정
- height: 픽셀이나 %로 크기지정
- hspace: Horizontal 여백
- vspace: Vertical 여백

▪ HTML4.01 -Table tag

➤ is a container tag that contains data in rows and columns

➤ <TR> Tag : 각 행을 만들

➤ <TD> Tag : 셀을 정의

➤ TR and TD are always enclosed inside <TABLE > Tags

➤ attribute

- align: 테이블 정렬 LEFT (default), RIGHT, CENTER
- valign: 테이블 내용의 수직 정렬 MIDDLE (default), TOP, BOTTOM
- border: 테이블 두께
- width: 테이블 너비 지정
- cellpadding: 셀 내용과 경계 사이의 여백
- cellspacing: 셀 사이의 간격
- rowspan: 2개 이상의 row를 합침
- colspan: 2개이상의 col을 합침

Nested Table : 테이블 안에 테이블

▪ HTML4.01 -List tag

➤ 리스트.

➤ : 태그로 식별되는 목록 요소라는 값 또는 텍스트 문자열이 포함되어 있음.

➤ Ordered: **integers**, **Roman numbers**) or **alphabets**로 표현됨

➤ Unordered : circular, square, or disk bullets로 표현됨

➤ attribute

- **type** : is to define the type of list element by choosing-Square, Circle, or **Disc(default)** for an unordered list - **A**(uppercase alphabet), **a**(lowercase alphabet), **I**(Big Roman numbers), **i**(Small Roman Numbers), or **1**(numbers, default)
- **start** : is to specify the **starting index of the ordered list**

▪ HTML4.01 - Form tag

➤ 입력 폼을 입력받음

➤ <FORM></FORM> paired tag and not a visible element in itself

➤ one webpage may contain multiple forms but nesting of forms is NOT allowed

➤ attribute

- **action** : destination web page로 받은 data를 제출함. 데이터 수신 파일 이름은 PHP나 ASP같은 것이어야 함
- **method** : 어떤 방식으로 보낼 것인가? The two possible values : GET and POST

➤ attribute (input tag)

- **type**: define the type of control to be added in the HTML form

'PASSWORD', 'CHECKBOX', 'RADIO', 'FILE', 'SUBMIT', 'RESET', 'BUTTON' 등등...

** One exception for Dropdownlist: use <select> & <option> tag instead of using input and type attribute

```
<H2> Type of Clothing</H2>
```

```
<SELECT>
```

```
<OPTION VALUE="TS">T Shirt</OPTION>
```

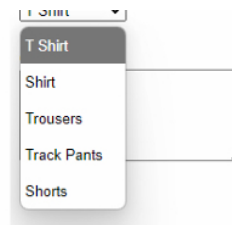
```
<OPTION VALUE="SH">Shirt</OPTION>
```

```
<OPTION VALUE="TR">Trousers</OPTION>
```

```
<OPTION VALUE="TP">Track Pants</OPTION>
```

```
<OPTION VALUE="SR">Shorts</OPTION>
```

```
</SELECT>
```



| Comment:

▪ Window Object

➤ Property

- history: Returns the History object for the window
- document: Returns the Document object for the window

➤ Method

- alert(): Displays an alert box with a message and an OK button
- close(): Closes the current window
- open(): Opens a new browser window
- confirm(): Displays a dialog box with a message and an OK and a Cancel button
- prompt(): Displays a dialog box that prompts the visitor for input
- focus(): Sets focus to the current window
- getSelection(): Returns a Selection object representing the range of text selected by the user

➤ Event

- onload: it occurs when an object has been loaded
- onunload: it occurs once a page has unloaded (or the browser window has been closed)

▪ Document Object

➤ Property

- document.getElementById(id): Find an element by element id
- document.getElementsByTagName(name): Find elements by tag name
- element.innerHTML=new html content: Change the inner HTML of an element

➤ Method

- `write()`: Write into the HTML output stream
- `element.setAttribute(attribute, value)`: Change the attribute value of an HTML element
- `focus()`: Sets focus to the current window
- `getSelection()`: Returns a Selection object representing the range of text selected by the user

➤ Event (Handler)

- `document.getElementById(id).onclick = function() {     code here ...     }`
 // Adding event handler code to an onclick event

➤ Property

- `title`: title of the document, `<title>` tag
- `location`: current location of the web browser
- `bgColor`: background color, “`bgcolor`” attribute
- `fgColor`: foreground or text color, “`text`” attribute
- `linkColor`: link color, `alinkColor`: active link color, `vlinkColor`: visited link color

➤ Event

- `onfocus` : an event occurs when an element gets focus
- `onblur` : an event occurs when an object loses focus
- `onkeydown`: an event occurs when the user is pressing a key
- `onkeyup`: an event occurs when the user releases a key
- `onclick` : an event occurs when the user clicks on an element
- `ondblclick` : an event occurs when the user double-clicks on an element
- `onmousedown`: an event occurs when a user presses a mouse button over an element
- `onmouseup`: an event occurs when a user releases a mouse button over an element

4. Web Security 1-2

HTTP : HTML 문서와 같은 리소스를 가져오기 위한 프로토콜
 웹에서 데이터 교환의 기초이며 수신자의 요청을 시작함.(web browser)
 전체 문서는 다양한 하위문서에 의해서 재구성됨.

HTTP 0.9(one-line protool) : 아주 간단. 1줄이고, GET 방식만 있음

HTTP 1.0(확장됨) : HTTP헤더가 요청, 응답 모두에 대해 도입됨

GET, POST, HEAD 메서드 지원

Content-type header(어떤 타입을 보낼지 헤더) 지원. 스크립트, 스타일시트, 미디어...

HTTP Status code

100~199 : Information : 101

200~299 : Success : 200(OK), 202(Accepted)

300~399 : 다른 URL에서 리디렉션됨

400~499 : Error : 401(Unauthorized), 403(Forbidden), 404(Not Found)

HTTP 1.1(표준화된 프로토콜. 일반적으로 많이 사용)

새로운 기능 추가됨 : 지속적인 커넥션 기능 추가. Keep-Alive header, 더 다양한 메서드 추가, Virtual hosting, 빠른 응답, 캐시 지원, HTTPS(Secure) 지원

HTTP 2.0

- * Stream Prioritization (스트링을 보낼 때 우선순위를 정함)
- * Server Push : 앞으로 필요할 것 같은걸 Push 해줌
- * Multiplexing : 한번의 TCP Connection으로 여러 파일 처리 가능
- * Binary framing layer

메시지를 전송하는 방법을 Text(ASCII)에서 Binary로 바꿨다. => 프레임 단위로 나누어 전송
1바이트에는 7비트만 데이터 저장. 1비트는

- * Header compression : 헤더를 정적 Huffman Code를 통해서 전송하는 크기를 줄임.
헤더 필드의 목록을 유지하고 업데이트해야함.

Encoding

Run-length encoding(RLE)

손실이 없는 압축법

반복되는 횟수를 지정하여 데이터를 압축하는 간단한 방법

[illegible]
$$12w \ 1b \ 12w \ 3b \ 24w \ 1b \ 14w \qquad 67\overline{w}|i|_{\underline{T}} > 18\overline{w}|i|_{\underline{T}}$$

wbwbw $\rightarrow$ 1w 1b 1w 1b 1w 5바이트 > 10바이트

주의 : 항상 용량이 줄어드는 것은 아님!

JPEG도 무손실 압축. FAX가 이런방식에 효율적. 연소되어 있는 것에 효율적

Huffman code

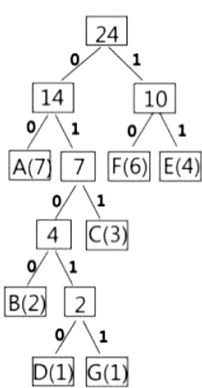
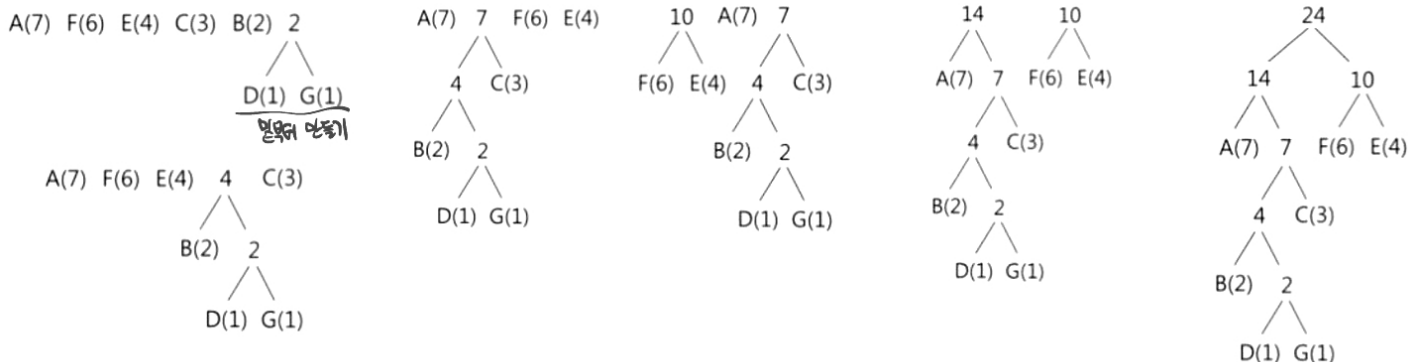
무손실 압축. 특정 유형의 optimal prefix code

AAAAAABBBCCCEEEEEFFFFFG



A(7) F(6) E(4) C(3) B(2) D(1) G(1)

Frequency occurrence with descending order



A: 00
F: 10
E: 11
C: 011
B: 0100
D: 01010
G: 01011

A(7) F(6) E(4) C(3) B(2) D(1) G(1)

Result:

AAAAAABBBCCCEEEEEFFFFFG

0000000000000000000001000100011011011010101111111110101010101001011

- 1991 - Tim Berners-Lee HTML 처음 만듦
- 1995 - HTML 2.0 첫 번째 공식 standard
- 1997 - 이미지 맵, CSS, font size & face
- 1999 - HTML 4.01 =>XHTML(XML 지원) 더 엄격한 버전
- 2012 - HTML5

HTML5

- * HTML 4와 XHTML을 모두 대체
- * 추가 플러그인 없이 대부분 디자인 가능
- * 애니메이션, 앱, 음악, 영화... 복잡한것들도 실행 가능
- * 크로스플랫폼

차이점

문법 간단화됨

새로운 미디어 element <audio>, <video>, <source>, <embed>, <track>

MP3, WAV, OGG (audio), MP4, WebM, OGG (video) IE는 wav,ogg지원X. Safari OGG 지원X

새로운 semantic/structural element <article>, <header>, <footer>, <nav>, <section>

없어진 element <big>, <center>, , <frame>, <frameset>, <strike>, <tt>, <applet>, etc

새로운 형태의 controls/tags (calendar, date, time, email, url, search)

<canvas> 2D 드로잉 - JS 통해서

로컬스토리지 지원

4. Web Security 2-1

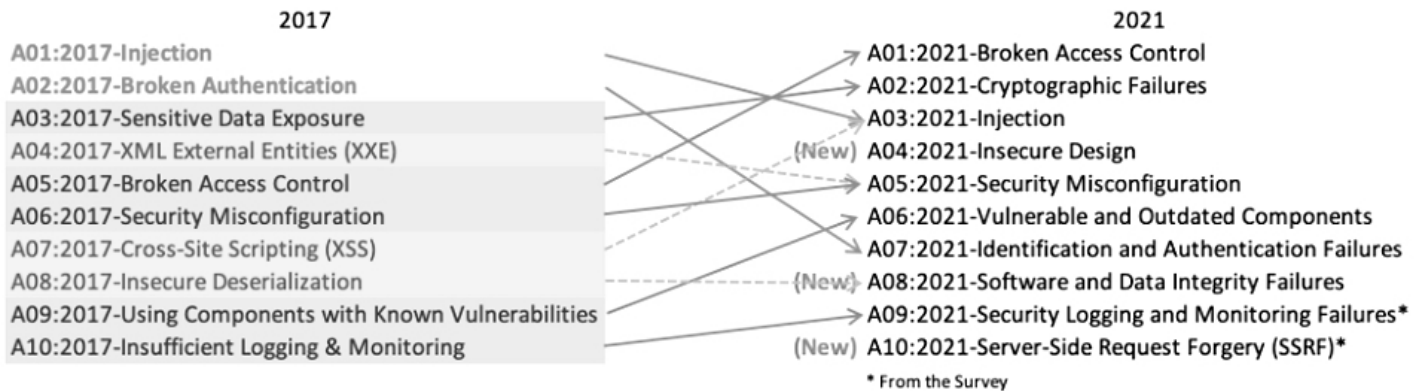
2004년부터 OWASP(Open Web Application Security Project) 발표

Top 10 for 2017 vs 2021

3개 새로운 카테고리

4개 네이밍, 범위조정

몇 개 통합됨



● Broken Access Control (5위 > 1위)

Access Control은 권한이 없는 사용자가 제어할 수 없도록 하는 것
무단 정보 공개, 데이터의 수정, 파괴 등으로 이어짐

Access Control includes...

- * HTML 페이지 수정, API 공격 도구 등을 사용해서 액세스 제어 검사를 우회
- *primary key를 바꿀 수 있게 하여 다른 사람의 레코드에 접근
- * 권한 높이기, 로그인 하지 않은채로 유저로 활동, 유저인데 admin처럼 행동
- * 메타데이터 조작 : JWT (JSON Web Token)를 무효화시키거나 조작

JWT vs Session vs Cookie

Cookie : Key-value string 유저의 기록을 추적하는데 도움.

Session : 유저 ID같은걸 더 안전하게 서버에 저장

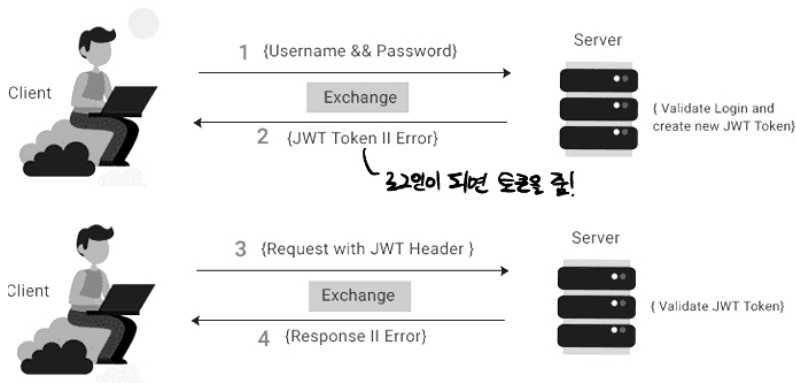
| Cookie | Session |
|----------------------------------|--|
| are stored on <u>client</u> side | are stored on <u>server</u> side |
| can store ONLY strings | can store objects |
| can be set to a long lifespan | when users close their browser, it also lost the session |

JWT(JSON Web Token) : 정보를 JSON 객체로 안전하게 전송하는 개방형 표준(RFC 7519)

JSON 문자열이 Base64URL로 인코딩됨.

Base64 Encoding XXXXX.YYYY.ZZZZ << 점으로 구분

Header Payload Signiture



Base64 Encoding

8비트를 6비트로 변환하는 인코딩 방식. 글씨를 6비트 안에 전부 표현

딱 들어맞지 않으면 padding은 "="로 채운다! 24비트씩 묶어야 해서 남은글자는 1~3개가 나올 수 있음.

Man => TWFu

Mana => TWuYQ==

| Threat Agents | Exploitability | Weakness Prevalence | Weakness Detectability | Technical Impacts | Business Impacts |
|----------------------|----------------|---------------------|------------------------|-------------------|-------------------|
| Application Specific | Easy: 3 | Widespread: 3 | Easy: 3 | Severe: 3 | Business Specific |
| | Average: 2 | Common: 2 | Average: 2 | Moderate: 2 | |
| | Difficult: 1 | Uncommon: 1 | Difficult: 1 | Minor: 1 | |

| Source | Text (ASCII) | M | | | | | | a | | | | | | n | | | | | | | | | | | |
|----------------|--------------|-----------|---|---|---|---|---|-----------|---|---|---|---|---|------------|---|---|---|---|---|------------|---|---|--|--|--|
| | Octets | 77 (0x4d) | | | | | | 97 (0x61) | | | | | | 110 (0x6e) | | | | | | | | | | | |
| | Bits | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | | | |
| Base64 encoded | Sextets | 19 | | | | | | 22 | | | | | | 5 | | | | | | 46 | | | | | |
| | Character | T | | | | | | W | | | | | | F | | | | | | u | | | | | |
| | Octets | 84 (0x54) | | | | | | 87 (0x57) | | | | | | 70 (0x46) | | | | | | 117 (0x75) | | | | | |

| Index | Binary | Char | Index | Binary | Char | Index | Binary | Char | Index | Binary | Char |
|---------|--------|------|-------|--------|------|-------|--------|------|-------|--------|------|
| 0 | 000000 | A | 16 | 010000 | Q | 32 | 100000 | g | 48 | 110000 | w |
| 1 | 000001 | B | 17 | 010001 | R | 33 | 100001 | h | 49 | 110001 | x |
| 2 | 000010 | C | 18 | 010010 | S | 34 | 100010 | i | 50 | 110010 | y |
| 3 | 000011 | D | 19 | 010011 | T | 35 | 100011 | j | 51 | 110011 | z |
| 4 | 000100 | E | 20 | 010100 | U | 36 | 100100 | k | 52 | 110100 | 0 |
| 5 | 000101 | F | 21 | 010101 | V | 37 | 100101 | l | 53 | 110101 | 1 |
| 6 | 000110 | G | 22 | 010110 | W | 38 | 100110 | m | 54 | 110110 | 2 |
| 7 | 000111 | H | 23 | 010111 | X | 39 | 100111 | n | 55 | 110111 | 3 |
| 8 | 001000 | I | 24 | 011000 | Y | 40 | 101000 | o | 56 | 111000 | 4 |
| 9 | 001001 | J | 25 | 011001 | Z | 41 | 101001 | p | 57 | 111001 | 5 |
| 10 | 001010 | K | 26 | 011010 | a | 42 | 101010 | q | 58 | 111010 | 6 |
| 11 | 001011 | L | 27 | 011011 | b | 43 | 101011 | r | 59 | 111011 | 7 |
| 12 | 001100 | M | 28 | 011100 | c | 44 | 101100 | s | 60 | 111100 | 8 |
| 13 | 001101 | N | 29 | 011101 | d | 45 | 101101 | t | 61 | 111101 | 9 |
| 14 | 001110 | O | 30 | 011110 | e | 46 | 101110 | u | 62 | 111110 | + |
| 15 | 001111 | P | 31 | 011111 | f | 47 | 101111 | v | 63 | 111111 | / |
| Padding | | = | | | | | | | | | |

위협이 되는 Agent : ex) 앱

Exploitability : 악용가능성이 있는가? 3-Easy

Prevalence : 만연한가? 일반적으로 존재하는가? 3-Widespread

Detectability : 취약성을 발견하기 쉬운가? 3-Easy

Technical impact 기술적 영향을 미치는가? 3-Severe

Business specific : 비즈니스에 미치는 영향

| Threat Agents / Attack Vectors | | Security Weakness | | | Impacts | |
|--------------------------------|-------------------|-------------------|------------------|--------------|------------|--|
| App. Specific | Exploitability: 2 | Prevalence: 2 | Detectability: 2 | Technical: 3 | Business ? | |

Attack scenarios

1. 프로그램이 확인되지 않은 데이터 사용

pstmt.setString(1, request.getParameter("acct")); << 공격자가 acct의 내용을 다른사람 이름으로 바꿈

ResultSet results = pstmt.executeQuery();

제대로 verify하지 않으면 공격자는 모든 사용자의 계정에 접근할 수 있음

http://example.com/app/accountInfo?acct=notmyacct <<다른사람 계정 접근 가능

2. 공격자가 target URL에 강제로 탐색함

admin 페이지 무단접근 가능

http://example.com/app/getappInfo

http://example.com/app/admin_getappInfo

Cryptographic Failure 암호화 실패 (3위 -> 2위)

| Threat Agents / Attack Vectors | | Security Weakness | | Impacts | |
|--------------------------------|-------------------|-------------------|------------------|--------------|------------|
| App. Specific | Exploitability: 2 | Prevalence: 3 | Detectability: 2 | Technical: 3 | Business ? |

* 이전 이름은 Sensitive Data Exposure. 앱이 데이터를 부적절하게 처리해서 민감 데이터 유출.

바뀐 이름은 암호화에 관련된 오류에 초점을 맞추기 위함. 이 문제는 데이터 노출과 시스템 손상으로 이어짐

Cryptographic Failure includes...

- * 일반 텍스트로 전송되는 데이터가 있는가? HTTP, SMTP, HTP 같은 프로토콜에 문제가 있나? 외부 인터넷 통시는 위험함. 모든 내부 트래픽을 검사하기.
- * 오래되거나 약한 암호화 알고리즘(DES, RC4, MD5 등)을 사용하는가?
- * 기본 암호화 키가 사용중이거나, 약한 암호화 키거나, 재사용되거나, 적절한 키 관리 및 교체가 안되었나?
- * 암호화가 제대로 되었나? 보안 지시문이나 헤더가 빠지지 않는가?

방지하는 방법

- * 중요한 데이터를 전송할 때 FTP, SMTP같은 legacy protocol를 사용하지 말기
- * 강력한 최신 표준 알고리즘을 사용(SHA-256, AES, SEED)
- * 전송중인 모든 데이터를 암호화하기. HSTS(HTTP Strict Transport Security) 사용
- * CSPRNG(암호화보안 의사난수생성기) 사용

Attack Scenarios

- #1 신용카드 번호같은걸 평문으로 저장하면 SQL 인젝션같은 방법으로 유출되었을 때 유출될 수 있음
- #2 TLS를 사용하지 않거나 약한 암호화를 사용하면, HTTPS->HTTP로 다운그레이드해 쿠키를 가로챌 수 있음. 이를 활용해 세션 탈취하고 데이터에 접근할 수 있음.
- #3 미리 해시값이 계산된 Rainbow table에 의해서 코드가 유추당할 수 있다. 이를 위해서 Salt 값을 추가해야 함. 또 간단한 hash 함수는 금방 뚫릴 수 있음.
- #4 plaintext로 된 로그인, 비밀번호, 쿠키에 저장된 세션ID가 탈취될 수 있음.

Injection 인젝션 (1위 -> 3위)

데이터 유효성 검사를 적절히 하지 않고, 잘못된 데이터 액세스 관행으로 공격자가 외부 코드를 삽입.

일어나면 안되는 DB 쿼리나 액션이 일어날 수 있음.

attack when...

- * 사용자가 제공한 데이터가 검증, 필터, sanitize되지 않았을 경우
- * 동적 쿼리나 context-aware escaping이 인터프리터에서 직접 사용될 때, 매개변수화된 쿼리를 사용해야함.
- * 적대적인 데이터가 ORM(Object-Relational Mapping) 검색 매개 변수 내 들어가 민감한 레코드 추출
- * 적대적인 데이터가 직접 사용되거나 concatenated(연결) 되는경우

방지하는 방법

안전한 API를 사용하기. 인터프리터X, 코드와 데이터가 분리되어 실행되도록. Django 같은 ORM 도구. 서버측에서 입력 유효성 검사.

특수 문자 이스케이프 사용 : 동적 쿼리에 인터프리터의 특정 이스케이프 문법을 사용하기.

LIMIT 및 기타 SQL의 제어를 사용하기

시나리오

```
#1 String query = "SELECT * FROM accounts WHERE custID='" + request.getParameter("id") + "'";  
    취약한 SQL 호출  
#2 Query = session.createQuery("FROM accounts WHERE custID='" + request.getParameter("id") + "'");  
    Query HQL
```

➤Retrieving hidden data 숨은 데이터 가져오기

★웹 Security 15-16 페이지 참고 ★

Injection : Cross-Site Scripting(XSS)

Cross-Site Scripting(XSS) : 악의적으로 스크립트를 심는 것

CSRF(Cross-Site Request Forgery) : 원치 않는 작업을 실행하도록 강제하는 공격.
공격자가 원하는 정보를 알아내고자 하는것

XSS를 막기 위해 - HTML character entities를 사용하기

"<" == < ">" == > " " == "&" == &

CSRF를 막기 위해 - 참조 검증하기 : 요청의 출처가 신뢰할 수 있는지 체크

<script> alert(document.referrer); </script>

요청에 CSRF 토큰을 포함시켜 유효성 검증

Insecure Design (4위, New)

앱의 수명 주기에서 가능한 한 빨리 pushing left(shifting left)하는 것을 강조하기 위해서

방지 방법

- * Appsec 전문가들에게 평가를 받고 디자인을 안전하게 만들기
- * 중요한 인증, access control, 비즈니스로직 등에 threat modeling 사용
- * 프론트에서 백엔드까지 타당성을 검증하기

Security Misconfiguration(6위 -> 5위)

시스템을 안전하게 설정하지 않은 것.

includes...

불필요한 기능이 활성화되거나 설치됨

default 계정이나 암호 사용

오류 처리가 스택 추적 또는 지나치게 많은 에러 메시지를 사용자에게 노출

업그레이드된 시스템은 최신 보안 기능이 비활성화되거나 안전하게 구성되지 않음

서버가 보안 헤더 또는 지시문을 보내지 않음

소프트웨어가 오래되거나 취약함

방지 방법

불필요한 기능, 구성요소, 문서 등 사용 최소화

업데이트, 패치 잘하기

segmentation(세분화), containerization(컨테이너화)을 하기, ACL 규칙 따르기

시나리오

- Directory listing option (Apache)

아파치에서 디렉토리 리스트를 다 보이게 하면 안됨! > Options Indexes SymLinks를 주석해제하기

XXE : XML External Entities

XML에서 외부 객체를 가져오는 기능인데, 취약점이 많이 발견되었음.

공격자가 서버 데이터에 접근하려고 할 수 있음.

file:///etc/passowrd

공격자가 서버의 private network에 접근하려고 할 수 있음.

https://192.0.0.1/private

끝없는 파일을 포함하여 DOS 공격을 할 수 있음.

file:///dev/random,,

Vulnerable and Outdated Components(6위)

이전 이름은 "알려진 취약점이 있는 컴포넌트 사용"

보안 결함이 있는 라이브러리, 프레임워크를 사용하는 것

Threat Agents / Attack Vectors		Security Weakness			Impacts	
App. Specific	Exploitability: 2	Prevalence: 3	Detectability: 2		Technical: 2	Business ?

includes...

사용중인 모든 컴포넌트에 대하여 모른다. 직접적으로 사용하는 것 + 중첩된(nested)된 것

소프트웨어가 취약하거나 지원이 중단된 경우.

적기에 플랫폼 프레임워크 등을 수정하거나 업그레이드 하지 않은 경우. 월별/분기별로 패치를 하는 환경에서 주로 발생. 그 안에 노출될 수 있음.

업데이트/업그레이드된, 시스템에서 호환성을 테스트하지 않은 경우 문제가 발생할 수 있음.

방지 방법

사용하지 않는 것, 불필요한 기능, 구성요소, 파일 등 삭제

버전 관리를 지속적으로 수행. OWASP Dependency check 같은 도구 사용

공식적인 소스에서 안전한 링크를 통해 component 얻기. 서명된 패키지 사용

유지보수되지 않거나 오래된 것에 대해서는 모니터링하기.

CVE list : 공개적으로 결함이 발견된 것들을 모아놓은 리스트

Identification and Authentication Failures(10위 -> 7위)

이전 이름 "Broken Authentication"

암호, 액세스 토큰 및 키와 같은 세션 제어 도구가 제대로 보호되지 않은 것

includes..

자동화된 공격(credential stuffing) 당함. 중첩된 의존성(nested dependency)를 포함.

약한 비밀번호, 기본 비밀번호

평문, 암호화되지 않은 데이터 저장소. 취약하게 해시된 비밀번호
MFA를 하지 않거나 효과가 없음
GET 메서드에 노출시키기
세션을 Reuse하기

방지 방법

MFA를 구현하여 credential stuffing, brute force, 도난당한 credential 재사용 공격 방지
로그인 후에는 내장 세션 매니저 사용 등...

Brute force attack ; 무단 액세스 권한을 얻기 위해서 사용자 이름과 암호를 추측하는 cracking 방법

Simple brute force attack : 외부 논리에 의존하지 않고 체계적으로 “추측”

Hybrid brute force attack : 외부 논리에서 시작하여 성공할 가능성이 높은 것부터 변형하여 시도

Dictionary attacks : 사전에서 가능한 문자열이나 구문을 찾아서 추측

Rainbow table attack : 암호화 해시 함수를 되돌리기 위해서 계산된 테이블

Reverse brute force attack : 비밀번호를 기준으로 아이디를 바꿔가면서 공격

Credential stuffing : 이전에 알려진 암호-사용자 쌍을 다른 웹사이트에 대해서 시도

방지 방법

Lockout policy : 로그인 시도 일정 횟수 시도 후 실패 후 계정 잠금

Progressive delays : 실패한 시도 후 일정시간 계정 잠금

강력한 비밀번호 요구 : 길고 강력한 비밀번호

Two-Factor Authentication : 여러 요소를 사용하여 액세스 권한 부여

Captcha : 시스템에 로그인하기 위해서 간단한 작업을 완료하게 하는 것

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 3	Prevalence: 2	Detectability: 2	Technical: 3	Business ?

Software and Data Integrity Failures (8위, NEW)

종속성 사용, 검증되지 않은 데이터, 외부 데이터의 불충분한 검증

includes...

신뢰할 수 없는 소스, CDN에서 플러그인, 라이브러리 모듈을 사용할 경우

안전하지 않은 CI/CD 파이프라인이 비인가 접근, 악성 코드 또는 시스템 손상을 초래 가능

충분한 무결성 검증을 하지 않고 앱에 적용

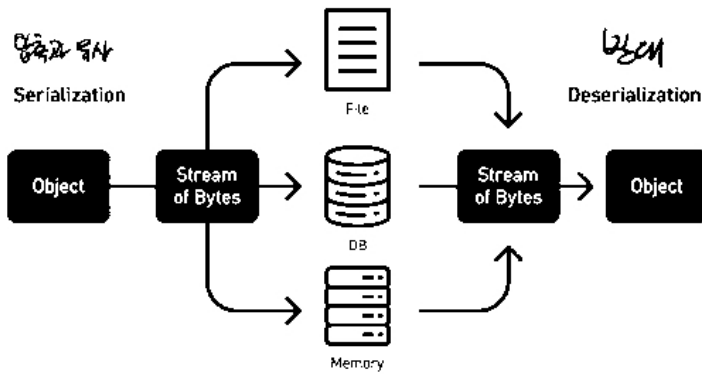
방지 방법

무결성을 확인하기 위해 디지털 서명 등 사용

NPM, Maven 같은 라이브러리와 종속성이 신뢰할 수 있는지 체크

Serialization : 데이터를 DB에 영구적으로 저장하는 방법

Deserialization : DB에서 원래 object로 다시 변환하는 방법



Serialization vs. Deserialization

python에서 `yalm.load()` 대신 `yalm.safe_load()` 사용하기

Security Logging and Monitoring Failures(9위)

이벤트나 의심스러운 action에 대해서 빠르게 잡아내지 못함.

includes...

감사할만한 이벤트가 로그되지 않음

경고와 오류가 없거나, 불충분하거나 불명확한 로그 메시지를 생성

앱과 API 로그가 의심스러운 활동을 모니터링하지 않음

앱이 실시간으로 공격을 탐지할 수 없음

예방 방법

모든 로그인, 접근제어, 서버사이드 입력 실패 기록을 로그될수 있게 하고, 충분한 기간동안 보관

로그가 로그관리 솔루션에서 쉽게 찾을 수 있게 만들기

로그데이터가 인젝션 등 공격을 방지할 수 있도록 암호화.

Server Side Request Forgery (SSRF) (10위)

앱이 사용자 입력 URL을 검증하지 않고 원격 리소스를 가져올 때 발생

클라우드에서 일어나는 신생 위협

6. Cryptography

XOR (Exclusive OR)

Input		Output
0	0	0
0	1	1
1	0	1
1	1	0

$$5 \oplus 7 = 101_2 \oplus 111_2 = 010_2 = 2$$

$$3 \oplus 10 = 0011_2 \oplus 1010_2 = 1001_2 = 9$$

$$X \oplus X = 0$$

Plaintext

사람이 이해할 수 있는 것. 영어 문장, 스크립트, 자바 코드

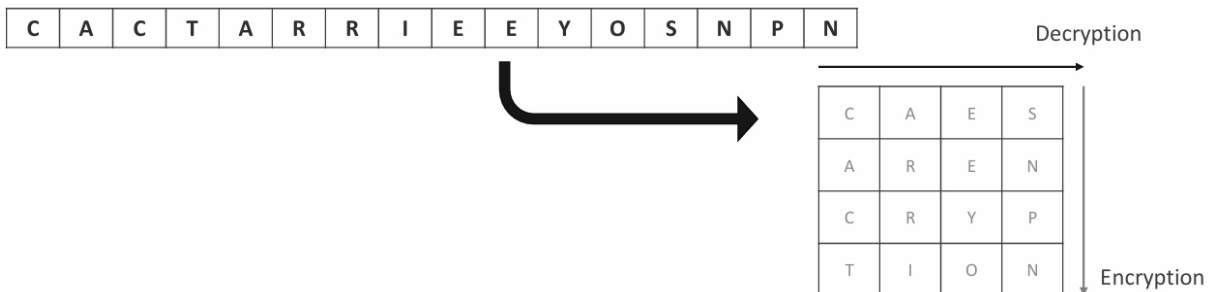
Ciphertext(Encrypted text)

사람이 이해 불가. 암호화 알고리즘을 활용해 plaintext를 암호화.

decryption 과정을 통해 원래 plaintext로 되돌림

Ancient cryptography

Scytale : 막대기에 감기



Caesar cipher

간단하다고 알려진 암호

substitution cipher의 일종. 일정 수만큼 위치를 옮김

(Encrypting) 왼쪽으로 세칸

(Decrypting) 오른쪽으로 세칸

RISQIIVBEJQIIQIYDQJEH > Be careful for assassinator

숫자로 대체해서 나타낼 수도 있음. A->0, B->1 +25

>>brute force에 취약

Monoalphabetic Substitution Cipher

Dancing Man Code (Sherlock Holmes) : 글자와 그림을 1:1로 대체함.

Random number table or Codebook : 21537 -> 215쪽에있는 37번째 글자

Ceasar와 달리 규칙이 없음

경우의 수 : 26! 하나를 찾아내면 그걸 제외하고 찾으려 됨.

빈도에 따라 유추 가능

Polyalphabetic Substitution Cipher

Vigenere cipher : 26*26의 표에서 해당 글자를 찾아내기. Plaintext와 Cipher Key의 교점이 것이 암호문
복호화도 똑같음.

Playfair cipher : 수동 대칭 암호. 'Charles Wheatstone'에 의해 발명. 'Lord Playfair'가 개선해서 진흥
IJ나 QZ를 같은 것으로 사용.

반복되는 것, 빈 글자(한글자로 끝나면)는 경우 X로 취급

KEY가 "playfairexample"이면 다음처럼 5*5가 만들어짐.

plaintext를 2글자씩 묶기. 반복되는 것, 빈 글자(한글자로 끝나면)는 경우 X로 취급.

P	L	A	Y	F
I	R	E	X	M
B	C	D	G	H
K	L	M	N	O
T	U	V	W	X

➤ Playfair cipher

• HI DE TH EG OL DI NT HE TR EX ES TU MP → BM OD ZB XD NA BE KU DM BE XM KU DM IF
BM OD ZB XD NA BE KU DM BE XM KU DM IF

(1)

P	L	A	Y	F
I	R	E	X	M
B	C	D	G	H
K	N	O	Q	S
T	U	V	W	Z

 HI
Shape: Rectangle
Rule: Pick Same Rows, Opposite Corners
BM

(2)

P	L	A	Y	F
I	R	E	X	M
B	C	D	G	H
K	N	O	Q	S
T	U	V	W	Z

 DE
Shape: Column
Rule: Pick Items Below Each Letter, Wrap to Top if Needed
OD

(3)

P	L	A	Y	F
I	R	E	X	M
B	C	D	G	H
K	N	O	Q	S
T	U	V	W	Z

 TH
Shape: Rectangle
Rule: Pick Same Rows, Opposite Corners
ZB

(4)

P	L	A	Y	F
I	R	E	X	M
B	C	D	G	H
K	N	O	Q	S
T	U	V	W	Z

 EG
Shape: Rectangle
Rule: Pick Same Rows, Opposite Corners
XD

(5)

P	L	A	Y	F
I	R	E	X	M
B	C	D	G	H
K	N	O	Q	S
T	U	V	W	Z

 OL
Shape: Rectangle
Rule: Pick Same Rows, Opposite Corners
NA

(10)

P	L	A	Y	F
I	R	E	X	M
B	C	D	G	H
K	N	O	Q	S
T	U	V	W	Z

 EX
Shape: Row
Rule: Pick Items to Right of Each Letter, Wrap to Left if Needed
XM

- (6) The pair DI forms a rectangle, replace it with BE
- (7) The pair NT forms a rectangle, replace it with KU
- (8) The pair HE forms a rectangle, replace it with DM

- (9) The pair DI forms a rectangle, replace it with BE
- (11) The pair NT forms a rectangle, replace it with KU
- (12) The pair HE forms a rectangle, replace it with DM
- (13) The pair MP forms a rectangle, replace it with IF

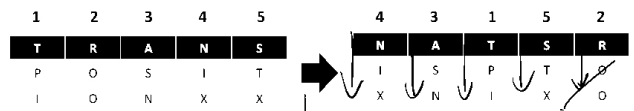
Transposition cipher 전치 암호

Simple transposition cipher : plaintext의 문자를 재정렬하기. 위치는 바뀌지만 글자 자체가 바뀌지는 않음.
빈글자는 X로 채움.

plaintext : TRANSPPOSITIPON

keyword : 43152

Simple Transposition Cipher
Plaintext: TRANSPPOSITIPON
Keyword: 43152
Ciphertext: NIXASNTPISTXROO



가로로 나열하고 뒤집은 후, 세로로 읽어서 cipher text 작성

Nihilist cipher : 가로로 있는 숫자를 보고 위에서부터 나열 후,

세로로 있는 숫자 순서대로 가로로 cipher문 작성

• Nihilist Cipher

Plaintext: this is good for secure cipher

Keyword: LEMON

Ciphertext: GSODOHTIISOFRSEPIHREUCRCE

→ 'thisi', 'sgood', 'forse', 'curec', 'ipher'

		L	E	M	O	N
		2	1	3	5	4
L	2	H	T	I	I	S
E	1	G	S	O	D	O
M	3	O	F	R	E	S
O	5	U	C	R	C	E
N	4	p	I	H	R	E

Modern Cryptography

Symmetric Key Cypptosystem 대칭형 암호시스템

Block cipher - 높은 보안 요구 ; 요즘에 많이 사용

Stream cipher - 블록보다 빠르고 secure한 특수한 상황에 사용

<-> Asymmetric Key Cryptosystem 비대칭형 암호시스템

== Public Key Cryptosystem.

Public Key와 Private Key 있음.

Confusion 혼돈

암호문과 평문 사이의 관계를 복잡하게 만들.

Diffusion 확산

평문의 비트가 암호문 전체에 퍼지도록 함. 한 글자가 바뀌면 연쇄적인 효과.

평문과 암호문 사이의 관계를 숨기려는 목적

비트 하나를 바꾸면 통계적으로 절반이 변경되어야 함.

보안은 이 성질을 만족해야 함

CIA + **Non-repudiation (부인 방지)** : 정보 송신자의 행위 부인 방지. 못 받았다고 하는걸 방지

Block Cipher (of Symmetric)

블록(고정된 길이의 비트 그룹)을 사용하는 방법

고정된 크기의 블록들을 동시에 처리하며, 한번에 한비트씩 처리하는 스트림 암호와는 반대.

최신 암호는 64비트(DES) 또는 256비트(AES)의 고정 크기 블록으로 데이터를 암호화되도록 설계.

Padding (패딩)

Bit Padding : 패딩할 부분의 첫비트(MSB)를 1로 하고 나머지는 0으로 채우기

*만약 패딩이 아니고 실제 데이터면 [1000 0000 0000 0000]를 추가하기

Byte Padding :

ANSI X923: 패딩을 0으로 채우고 나머지 비트에 패딩인 것의 비트수만큼 수를 쓰기 00 00 00 04

PKCS #7: 04 04 04 04 이렇게 쓰기.

만약에 평문이 블록사이즈와 정확히 일치하면 [08 08 08 08 08 08 08 08]을 추가하기

Electronic Code Book (ECB) : Symmetric key encryption에서 대부분 쓰이는 방법.

각 블록은 동일한 메시지 복을 가지고 암호화 됨. 항상 동일하게 암호화됨.

replay attack에 취약 : 4567이 뚫라면 456745674567도 뚫림.

Cipher Block Chaining(CBC) : ECB의 단점을 보완. Encryption을 하고 XOR을 한 후에 다음 알고리즘에 대한 입력으로 제공됨

Stream cipher (of Symmetric)

더 보안이 필요한 특수한 환경에서 사용. Seed값을 공유하고 서로 그 Seed값으로 각자 암호를 생성한 후

$C = P \oplus X$ (Encrypt)

$C \oplus X = P$ (Decrypt)

DES(Data Encryption Standard)

64비트 plaintext를 64비트 cipher로 암호화하는 symmertric 암호화이다.
NIST에 의해 발표되어 1976~2002동안 표준 블록 암호화로 사용됨

1단계 : 첫 번째 단계에서 64비트 일반 텍스트는 초기 순열(비트 재배열)하여 왼쪽 일반 텍스트(LPT 32비트) 및 오른쪽 일반 텍스트(RPT 32비트)라고 하는 두 개의 32비트 순열 블록을 생성합니다

2단계 : 이제 56비트 키를 사용하여 이 LPT 및 RPT에서 16라운드의 DES 암호화가 수행됩니다

3단계 : 16라운드 후 32비트 LPT와 32비트 RPT가 통합되어 다시 64비트 블록을 형성한 다음 이 64비트 블록에 최종 순열을 적용하여 64비트 암호문을 얻습니다.

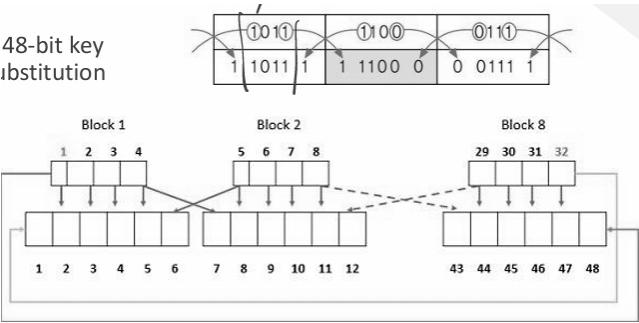
[2단계에 대한 자세한 설명]

1. Key Transformation

- 초기 키는 64비트.
- Permuted choice(순열 선택)을 거쳐 56비트로 만들기
 - : 각 블록에서 8번째 비트인 parity(오류검증) 비트 1개씩 제외하고 7비트만 사용
- Left-Circular Shift : 다시 반으로 나누어 28비트씩을 각자 왼쪽으로 한비트 이동함. 끝은 끝으로.
- Permuted choice(순열 선택)을 거쳐 다시 48비트로 만들기

2. Expansion Permutation

- 32비트 RPT를 8개의 4비트 블록으로 나누어져 있다.
- 6비트로 확장해야 함.
- 4비트는 그대로 내려오고, 양끝의 한 비트는 이웃으로.
- =>48비트로 확장됨!
- 48비트 RPT는 48비트 key와 XOR되고,
- S-Box substitution으로 보내짐.

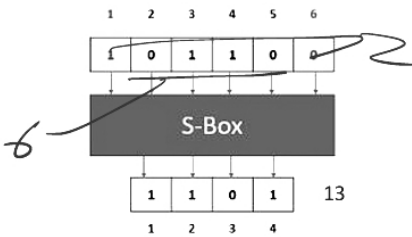


3. S-box Substitution

- 입력된 48비트 블록은 8개의 6비트로 나뉘며 S-box라고하는 8개의 substitution box에 들어감.
- 6비트로 들어간 비트는 4비트로 나옴.
- 각 S-box는 0-3의 행, 0-15의 열인 테이블로 간주됨
- 첫비트와 마지막비트는 행, 가운데 4개는 열 테이블을 보고 거기에 있는 숫자를 2진수로 표현 한 것이 결과.
- 따라서 4비트 8묶음 32비트가 다음 단계로 보내짐.

	0	1	2	3	4	5	6	7								
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	2	6	13	11	15	12	9	7	3	10	5	0
	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

S-Box Table 1



4. P-box Permutation

- 32비트 입력이 순열(순서바꿈)되어서 다음단계로 전달됨.

5. XOR and Swap

32비트 RPT는 바뀌어왔지만 LPT는 그대로임.

여기서 LPT와 P-box에서 나온 RPT를 XOR => New RPT

처음 RTP 값 => New NPT임

>>>> 이 과정을 16번 반복.

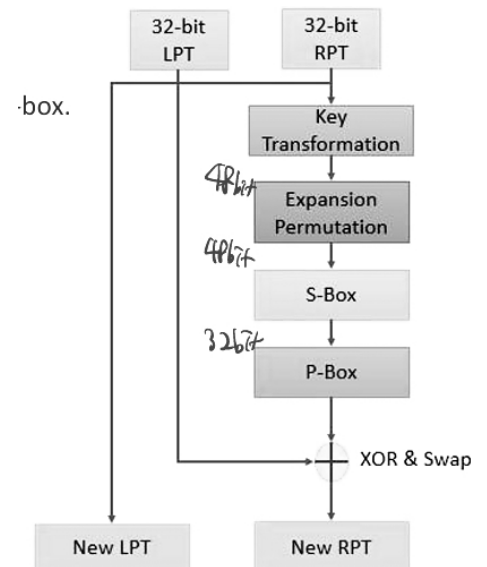
Initial Permutation 과 final Permutation은 역함수 관계이다!

Double DES : DES 두 번 반복.

암호 > 암호

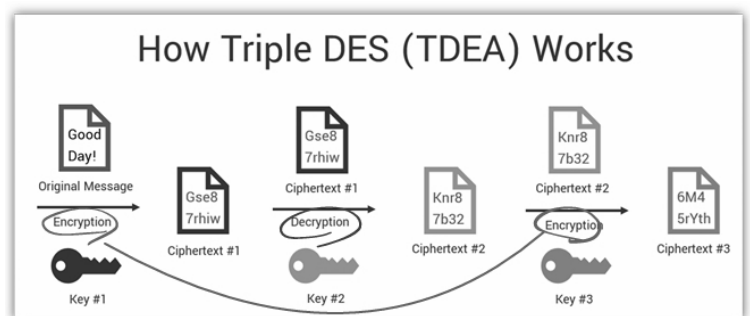
Triple DES : DES를 세 번 반복.

암호 > 복호 > 암호



3개중에 하나 선택 가능

1. 세 번 다 키 다르게
2. 드개는 같고 하나는 다르게
3. 세 번 다 같게



AES(Advanced Encryption Standard)

공모 결과 2001년에 NIST에서 도입

Symmetric Block cipher. DES의 약점 극복

128비트 키와 128비트 평문을 이용해 128비트 암호문 생성

	Key Length	Block Length	# of Round
AES-128	4 byte	4	10
AES-196	6	4	12
AES-256	8	4	14

1단계 : 처음에는 16바이트 키 or 4개 단어 키를 => 4바이트 키 44단어로 확장.

2단계 : 암호화 프로세스의 제일 첫 단계에서 16바이트 평문(또는 4단어 평문)은 4단어 키(W0,W1,W2,W3)과 XOR 이것이 첫 번째 라운드로 들어감

3단계 : 첫 라운드에서 XOR의 결과가 Substitution Bytes, Shift rows, Mix column, Add round key 함수에 의해 순차적으로 진행됨.

Add round key 단계에서 Expand key에서 새로운 4단어 key를 받음(W4,W5,W6,W7).

add round key 한 결과가 2라운드로 전달됨.

함수들 : RotWord, SubWord, Rcon

Substitute Bytes

Substitute로 들어온 입력은 4x4 matrix가 된다. 입력이 16바이트이므로 각 element가 1바이트이다.

AES에는 S-Box라고 불리는 16x16 matrix가 있는데, 8비트씩 256개이다.

1단계) 입력 값의 element의 왼쪽 4비트는 행, 오른쪽 4비트는 열을 결정함.

2단계) 이 값은 S-Box에서 새 값을 가져오는 인덱스 역할을 함. 해당하는 값으로 변경.

Shift Rows

4x4 matrix로 전달된다.

왼쪽으로! 움직인다.

첫 번째 row는 아무것도 하지 않음

두 번째 row는 왼쪽으로 1바이트 shift

세 번째 row는 왼쪽으로 2바이트 shift

네 번째 row는 왼쪽으로 3바이트 shift

>> Mix columns로 이동

Mix Columns

미리 정의된 상수 행렬을 곱한다.

행렬곱한 값.

>> 결과가 add Round key로 이동

Add round key

현재 상태 행렬과, key를 XOR.

column-wise 함수이다. byte-level 함수이다.

6. Cryptography2

Symmetric key 암호시스템의 키 배포 문제

보내는 사람과 받는사람 모두 암호를 사용하기 전에 secret key를 서로 공유하여야 함.

secret key 배포는

확장가능하지 않음(not scaleable). 1000명의 사람들끼리 통신하기 위해서 모두가 각자 다른 999개 키 필요

>> Assymetric key가 제안됨.

Diffle-Hallman(DH) Key change

Assymetric 부분: Diffie-Hellman 키 교환 프로토콜 자체는 공개 매개변수와 개인 키를 사용하여 공통의 비밀 키를 생성

Symmetric 부분: 생성된 공통 비밀 키는 대칭 암호화 알고리즘에 사용되어 실제 데이터의 암호화/복호화를 수행

Alice : **a**, g, p, A, B

Bob : **b**, g, p, A, B

a와 b는 서로에게 보내지지 않으며, private하고 random함

Alice와 Bob은 DH 알고리즘을 통해 알려진 parameter들을 이용해 각각 계산하여 K를 얻음.

K는 두 당사자가 같으며, key 역할을 함.

가로채도 불완전한 정보 세트만 가져갈 수 있음.(g, p, A, B)

a와 b는 전송 프로세스에 전혀 관여X, 인터셉터가 K를 얻을 수 없어 해독 불가능.

DH의 과정

p=23, g=5, a=5(Alice의 private key), b=15(bob의 private key)라 하자. 실제로는 크고 소수인 값 사용.

➤ **Step1:** Both (Alice & Bob) know the value of p & g

➤ **Step2(Calculate A):**

$$\bullet A = g^a \% p \text{ (Alice } \rightarrow \text{ Bob)} = 5^5 \% 23 = 15,625 \% 23 = (679 * 23 + 8) \% 23 = \underline{8}$$

➤ **Step3(Calculate B):**

$$\bullet B = g^b \% p \text{ (Bob } \rightarrow \text{ Alice)} = 5^{15} \% 23 = 30,517,578,125 \% 23 = (1,326,851,222 * 23 + 19) \% 23 = \underline{19}$$

➤ **Step4(Calculate Alice's Secret Key using Alice's Private Key a):**

$$\bullet K = B^a \% p = 19^5 \% 23 = 47,045,881 \% 23 = \underline{2}$$

➤ **Step5(Calculate Bob's Secret Key using Bob's Private Key b):**

$$\bullet K = A^b \% p = 8^{15} \% 23 = 35184372088832 \% 23 = \underline{2}$$

➤ **Step6:**

• Both Alice and Bob calculated and shared 'K' and can encrypt or decrypt the raw data

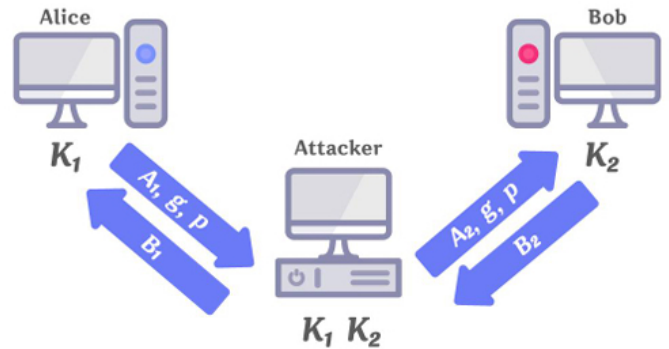
$$A \text{ 계산} = g^a \% p$$

$$a \text{를 이용한 K 계산} = B^a \% p$$

K는 공유한다!

DH에서 **Man-in-the-middle**

Eavesdropper(attacker) 공격자 Eve는 중간에서 a,b를 알아내서 양쪽방향의 K를 모두 알아내버림.



RSA(Rivest, Shamir, Adleman) - 1978

현재 제일 널리 쓰이는 public-key 암호시스템
암호화 키는 public이지만, 복호화 키는 private.

Scenario #1(Confidentiality): public key로 RSA 암호화, private key로 복호화(받는사람만 볼 수 있게)

Scenario #2(Non-repudiation): private key로 RSA 암호화, public key로 복호화(부인 방지)

내가 암호화 안했는데? 할 수 없음. private key로 암호화를 했기 때문. 법적 근거로 사용가능

public key와 private key는 unique한 쌍

➤ **Step1:**

- Choose two distinct prime numbers p and q (For security purposes, p and q should be chosen at random)

➤ **Step2(Compute n):**

- Compute $n = pq$ (n is used as the modulus for both public & private keys); $n = 61 * 53 = \underline{3,233}$

➤ **Step3(Compute L using Least Common Multiple):**

- L : Least Common Multiple of $(p-1)$ and $(q-1) = \text{LCM}(60, 52) = \underline{780}$

➤ **Step4(Find/Choose E using L):**

- $1 < E < L$ (E & L are coprime: 서로소); $1 < E < 780$ (coprime of 780: 17, 19, ...) Let $E = 17$ or 19

➤ **Step5(Find/Choose D using E & L):**

- $1 < D < L$ ($1 = (E * D) \% L$); $D: 413$ (if $E: 17$) or $D: 739$ (if $E: 19$); $\underline{1 = (17 * 413) \% 780}$

➤ **Finally,**

- Public key (E, N) : $(17, 3233)$ or $(19, 3233)$
- Private key (D, N) : $(413, 3233)$ or $(739, 3233)$

➤ **Encryption**

- Encryption function $c(m) = m^e \% n$ (m : plaintext);
- $c(m) = m^{17} \% 3233 \Rightarrow 65^{17} \% 3233 = \underline{2790}$ (if $m=65$)
- $c(m) = m^{19} \% 3233 \Rightarrow 65^{19} \% 3233 = \underline{232}$ (if $m=65$)

➤ **Decryption**

- Decryption function $m(c) = c^d \% n$ (c : encrypted ciphertext);
- $m(c) = c^{413} \% 3233 \Rightarrow 2790^{413} \% 3233 = \underline{65}$
- $m(c) = c^{739} \% 3233 \Rightarrow 232^{739} \% 3233 = \underline{65}$

Key Differences	Symmetric	Asymmetric
Purpose	Encryption	Key Exchange
Key Lengths	128 or 256-bit key size	RSA 2048-bit or higher key size
Number of keys	Uses a single key as a secret key	Uses two keys (Private & Public)
Speed	Fast	Slower than symmetric
Data size	Used to transmit big data	Used to transmit small data
Algorithms	AES, DES, 3DES	RSA, Diffie-Hellman
Security properties	Confidentiality	Confidentiality, Non-repudiation, Authentication
취약점	Key 공유 과정	Man-in-the-middle(DH)

HASH(함수)

임의의 형태의 데이터가 Hash 함수를 지나면 임의의 고정된 크기의 값으로 return됨
 해시함수에 의해 반환된 값을 hash value, digests, simply hashed라고 부름

문제점

- 인식가능성 : weak한 암호는 reverse generater에 의해 hash값이 알아차려질 수 있음.
- 속도 : 해시는 처리 속도가 빨라 임의의 digest와 해킹하려는 digest를 비교하기 용이함.

Rainbow Attack(미리 계산된 공격)을 방지하는 방법

- Key Stretching 키 늘리기 : digest를 다시시킴. hash함수를 여러번 계속하게 해서 계산 시간을 늘림.
- Salting : 원문에 임의의 값을 암호화하려는 평문과 함께 저장. salt는 충분히 커야 함. 아니면 rainbow당함

해시함수의 특성

- * Preimage Resistance(역상 저항성; 일방향성) : 해시값을 보고 원래 값을 찾는 것이 어려워야 함.
- * Second-preimage resistance(제2 역상저항성, **약한** 충돌저항) : 같은 해시값을 출력하는 다른 input을 찾기 어려워야 함. 한쪽은 고정
- * Collision Resistance(충돌저항성; **강한** 충돌저항) : 동일한 출력을 값는 입력이 있어서는 안된다. 양쪽 모두 알지 못함
- * Avalanche Effect(눈사태 효과) : 조그마한 변화도 반 이상의 비트가 바뀜.

Birthday Paradox(생일 역설) : 상대적으로 적은 수의 입력 값만으로도 해시 값 충돌이 발생할 확률이 높음
 >> 충분히 긴 해시 함수를 써야한다

모듈로256은 값이 256이되면 0이 됨. > 엄청오래걸림

해시함수의 적용

MD5

- 1991년 MD2, MD4를 대체하기 위해서 설계됨
- 1996년 설계 결함 발견
- 구형 시스템에서는 여전히 사용. 128비트 해시값 생성

SHA1 & SHA2(Secure Hash Algorithm)

- 미국 NSA에 의해 설계됨
- SHA1 : 160비트 해시값 생성
- SHA2 : 224, 256, 384, 512비트가 있음 **SHA-256**(256비트 결과)이 대중적

MAC(Message Authentication Function)

- : 메시지를 인증하는데 사용되는 간단한 정보(tag라고도 함)
- 서로 미리 공유된 Key를 사용. 데이터의 integrity와 authenticity 보장

HMAC(Hash-based MAC) : 해시기반 MAC

MAC/HMAC의 한계

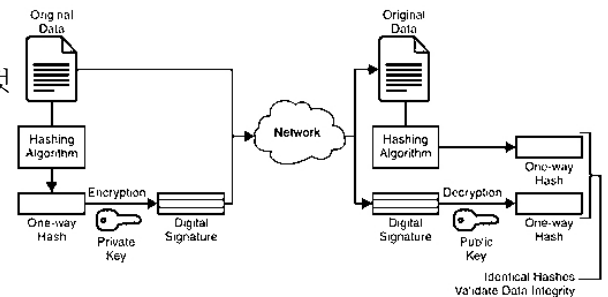
- 키 배포 문제 : 미리 키를 공유해놔야 함.
- 제 3자에 대한 인증 불가 : 제 3자는 MAC 계산 불가.
- No-Repudiation 불가 : Repudiation을 보장할 수 없음.
- Replay attack : 한번 가로챈 데이터를 다시 사용해서 공격할 수 있음 ex) 은행공격
- >>방지 방법 : 시퀀스 번호 보내기, 타임스탬프(시간정보 추가), Nonce(비표) : OTP 사용

Digital Signature

:디지털 메시지 또는 문서의 진위여부를 파악하기 위해서 만든 것
유효한 디지털 서명은 Authenticity, Integrity를 보장함.

일반적으로 세 가지 알고리즘으로 구성됨

1. 키 생성 알고리즘 > public, private key를 출력함
2. 메시지와 public키가 주어지면 생성하는 알고리즘
3. 메시지, public 키, 서명이 주어지면 authenticity를 검증하는 알고리즘



공개 키의 문제점

- 공개 키가 실제로 sender의 것이 맞는가? man-in-the-middle의 위험 있음
- RA(Register Authority)이 인증서를 등록하게 해 줌
- CA(Certification Authority) 인증을 하는 기관...

6.Crtptography2 마지막 페이지 그림 보기!

6. AI Security

AI

2020년에 시스템에 문제가 생겼다는데 평균 207일 걸림, 평균 처리 기간 73일 => 280일

IBM Deep Blue : 1997 체스봇

IBM Watson : 2011 natural language로 퀴즈

AlphaGo : 2016 바둑

자율주행차 aka Autonomous Vehicle(AV)

레이더, LiDAR, Sonar, GPS 등...

음성인식 : ASR(Automatic Speech Recognition), STT, TTS

Stable diffusion AI(이미지 생성) : 컴퓨터비전 + NLP(자연어처리)

3가지 종류의 AI

ANI(Artificial Narrow Intelligence) - Weak AI

AGI(Artificial General Intelligence) - Strong AI aka Deep AI 영화HER

ASI(Artificial Super Intelligence) - Super AI

AI > Machine Learning(1980) > Deep Learning(2010)

Traditionadl ML vs DL

Artificial Neural Networks(ANNs) : 뉴런을 모방.

SNN(Simple Neural Network)

DLNN(Deep Learning Neural Network) : 여러 층으로 딥러닝

ML : Feature Engineering 과정이 있음. 전문가가 데이터를 전처리

DL : 전처리

AI의 역사

1950 : Turing Test by **Alan Turing**

1956 : AI 용어가 만들어짐

1960s : 로봇 만들어짐, 1964 MIT에서 ELIZA 챗봇, 1966 Stanford 모바일 로봇

1971~1990 : 암흑기

1997 : IBM DeepBlue 체스

1998: KISMAT 감정로봇

1999 : 소니 아이보

2009 : 구글 자율주행차

2011 : 애플 시리, IBM Watson

1st Boom

John MacCarthy => 'AI'용어 만들.

정의 : System that act like humans
 System that think like humans
 System that act rationally
 System that think rationally

Turing test : 판정자에게 응답을 한 것이 컴퓨터인지 사람인지 맞춰야 함.

>> 컴퓨터를 인간으로 오인하면 튜링 테스트 통과

The immitation game

The Chinese room argument (by John Searle)

컴퓨터가 어떤 언어를 이해하는 것처럼 보일 수 있지만, 실제로는 단순히 규칙에 따라 기계적으로 작업을 수행하는 것일 뿐이며, **진정한 이해를 가지고 있는 것은 아니다**

1st winter

제한된 컴퓨터 파워

상식에 대한 지식?

조합의 폭발 >> 투자가 끊김

Boom(1980-1987)

expert system의 등장

정보혁명

Rule-based expert system : 하나하나씩 다 룰을 만들어서 떠먹여줘야함.

2nd winter(1987-1993)

2nd Boom(1993-2011)

딥러닝, 빅데이터, AGI (2011~현재)

ILSVRC(Imagenet Large Scale Visual Recognition Challenge)

2012년에 딥러닝 도입, 2015년에 휴먼에러 넘어섬

머신 러닝의 3가지 종류

Unsupervised Learnign 비지도학습: 레이블 없음

Supervised Learning 지도학습 : 레이블 있음

Reinforcement learning 강화학습: 보상을 최대화하는 학습방법

Supervised VS Unsupervised => 정답이 있음

Supervised : Classification vs Regression 회귀

Unsupervised : Clustering(유사한것끼리 묶기), Dimensionality Reduction 차원축소(데이터의 주요 특징을 추출해서 차원을 줄임)

ANN (Artificial Neural Network)

Input layer(pixel) -> hidden layer 몇 번을 거쳐서 -> output

Anomaly Detection 이상감지

산업 이상감지

의학 이상감지

소셜미디어 이상감지

사기감지

Anomaly vs Novelty vs Outlier

Anomaly : 이상한 것

Novelty : 비슷한데 기존에 있을 수 없는 새로운 특성

Outlier : 엄청나게 떨어진 값.

Deep fake

GAN 사용

XAI (Explainable AI)

인간이 이해할 수 있는 AI. <-> Black Box 개념과 대조

Beer and Diapers 월마트

맥주와 기저귀를 같이 배치했더니 매출이 늘어남.

Amazon

Anticipatory shipping 특허

넷플릭스

Quantum theory

EDA(Exploratory Data Analysis)

데이터를 분석하고

Messy data vs Tidy data

Tidy Data (by Hadley Wickham)

- * 각각의 변수는 한 column을 구성
- * 각각의 관측치(값)은 한 row를 구성
- * 각각의 관측치는 table을 구성

pivot vs melt

column에는 값이 들어가면 안 됨.

한 column에는 한 값만...

PDF 참조

Visualization 시각화 -- Kaggle

Comparison : 비교

Distribution : 분포

Composition : 구성

Relationship : 관계

Linear regression 선형 회귀 - 값을 예측

AI와 보안

AI에서 중요한 것은? - 데이터와, 데이터 모델

Data Privacy

Clearview 얼굴인식 스캔들

딥페이크

중국의 대량 감시

Privacy

Brandeis and Warren이 "혼자 있을 권리"라는 의미로 1890년 명명

다양한 의미를 가질 수 있음 : 제한된 액세스, 비밀, 개인정보 관리, 인격 보호, 친밀도

=> De-identification processing이 개인정보와 프라이버시를 지키는 데 중요하다.

De-identification techniques

redacting information(데이터 수정), 사진이나 비디오. Masking, Blurring, Plexation, Warping, Inpainting
데이터 집계

micro-aggregating : 평균 내버리기 등

removing some variables : 변수 지워버리기

coding or pseudonymization(코딩, 가명화)

generalizing : 일반화하기 1985년생 -> 1980년대생

masking : 가리기. 주민번호 뒷자리 가리기

data-swapping : 비슷한 데이터로 바꾸기

differential privacy : 약간의 노이즈 주기

encryption/hashing ; 암호화하기

Data Corruption & Data Poisoning

Integrity(무결성)와 reliability(신뢰성)를 보장해야 함

Data Poisoning : 소량의 데이터를 섞으면, 의도치 않은 해로운 결과를 불러일으킬 수 있음.

인식 가능한 공격

챗봇의 차별적인 발언

Anomaly detection : 실시간으로 이상한 데이터들을 골라내야 함.

Evasion attack (Adversarial attack - Imperceivable attack type)

공격적인 예시를 주면 분류에 혼동을 줌

White box attack : 모델에 대한 모든 내용을 알고 있을 때 가능한 공격 (FSGM)

<-> Black box attack : 모델에 대한 내용을 보른채로 공격

Non-Adaptive Black-Box Attack: 데이터 분포만을 알고, 모델 출력 값에 직접 접근할 수 없는 상태에서의 공격.

Adaptive Black-Box Attack: 모델에 쿼리를 보내 입력을 변경하면서 출력 값을 얻을 수 있는 공격.

Strict Black-Box Attack: 정해진 입력-출력 쌍을 알 수 있지만, 입력을 변경하여 출력을 관찰할 수는 없는 공격.

적대적 공격의 목적

Confidence Reduction(신뢰도 감소) : 신뢰도를 떨어뜨림

Misclassification(오분류) : 다른 무언가로 잘못 인식시키게 함

Targeted Misclassification(타겟된 오분류) : 무언가를 특정한 신호로 인식시키게 함

Source/Target Misclassification(소스/타겟 오분류) : 1:1로 특정 값을 특정 값으로 인식하게 함

Fast Gradient Sign Method (FSGM)

Loss 값을 최대화하여 output을 출력함.